arXiv:2402.18734v1 [cs.LG] 28 Feb 2024

Priority Sampling of Large Language Models for Compilers

Dejan Grubisic *
Rice University, Meta AI

Chris Cummins
Meta AI

Volker Seeker
Meta AI

Hugh Leather
Meta AI

Abstract

Large language models show great potential in generating and optimizing code. Widely used sampling methods such as Nucleus Sampling increase the diversity of generation but often produce repeated samples for low temperatures and incoherent samples for high temperatures. Furthermore, the temperature coefficient has to be tuned for each task, limiting its usability. We present *Priority Sampling*, a simple and deterministic sampling technique that produces unique samples ordered by the model's confidence. Each new sample expands the unexpanded token with the highest probability in the augmented search tree. Additionally, Priority Sampling supports generation based on regular expression that provides a controllable and structured exploration process. Priority Sampling outperforms Nucleus Sampling for any number of samples, boosting the performance of the original model from 2.87% to 5% improvement over -Oz. Moreover, it outperforms the autotuner used for the generation of labels for the training of the original model in just 30 samples.

1 Introduction and Motivation

Large language models (LLMs) have proven their ability in the software engineering domain to generate the code and documentation [16, 2], translate code between programming languages [13, 3], write unit-tests [8, 23], as well as detect and fix bugs [1, 29]. CodeLlama [22], ChatGPT [20], and Codex [5] excel in various code manipulation tasks significantly improving the coding experience. Some of the models such as AlphaCode [17] are pretrained on competitive programming tasks which enables the model to optimize code on the source level for several languages.

All these models could boost LLM's performance by generating an ensemble of diverse solutions, from which we evaluate and choose the best. This is usually done by increasing the entropy of generation [21, 15], or expanding the search tree [30, 12, 26, 24]. Sampling also enables us to better understand the capacity of the LLM on a given task and the range of possible solutions. This is particularly important in code generation where generating a variety of responses can be valuable in exploring different implementation ideas.

Current sampling approaches have few major problems. Temperature-based sampling [21, 15] requires a significant amount of computation to find the optimal temperature. The optimal temperature may also depend on the context, which requires additional evaluations. Once we set the temperature, sampling often produces a large number of duplicates and semantically meaningless answers, wasting available samples.

To motivate our approach, we show the average number of unique samples generated by Nucleus Sampling compared to Priority Sampling (Figure 1). On 50k test examples, Nucleus Sampling generates less than 5 unique examples on average for 100 samples, while Priority Sampling generates 100. The reason for this is that Nucleus Sampling either chooses the high probability generation repeatedly or it outputs non-coherent output which we don't count as a meaningful unique sample.

*Corresponding author: dx4@rice.edu

Preprint. Under review.

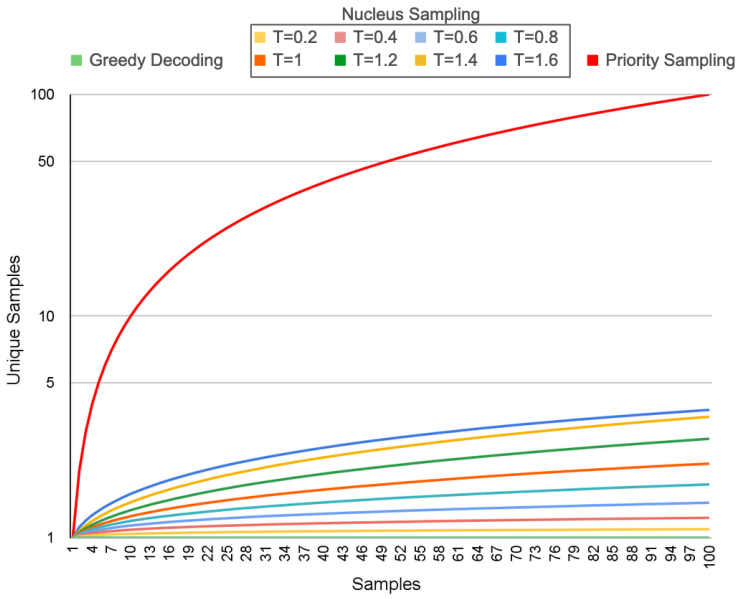


Figure 1: Average number of unique samples generated from 50k unseen test programs. Priority sampling produces a higher ratio of unique samples than nucleus sampling.

In this work, we present Priority Sampling, a deterministic sampling technique that guarantees unique samples, for which the model has the highest confidence. Furthermore, we guarantee that produced samples will adhere to the regular expression (inspired by Willard [28]), which is particularly important for code generation and code optimization.

We evaluate Priority Sampling on the task of optimizing LLVM optimization passes [7] in which the model is trained to predict optimizations found by the long-running autotuner. Priority Sampling outperforms Nucleus Sampling [21] for any number of samples, reaches 91% of the autotuner improvement over -Oz in just 5 samples, and even outperforms the autotuner used to generate labels for finetuning the original model with 30 samples (Figure 3).

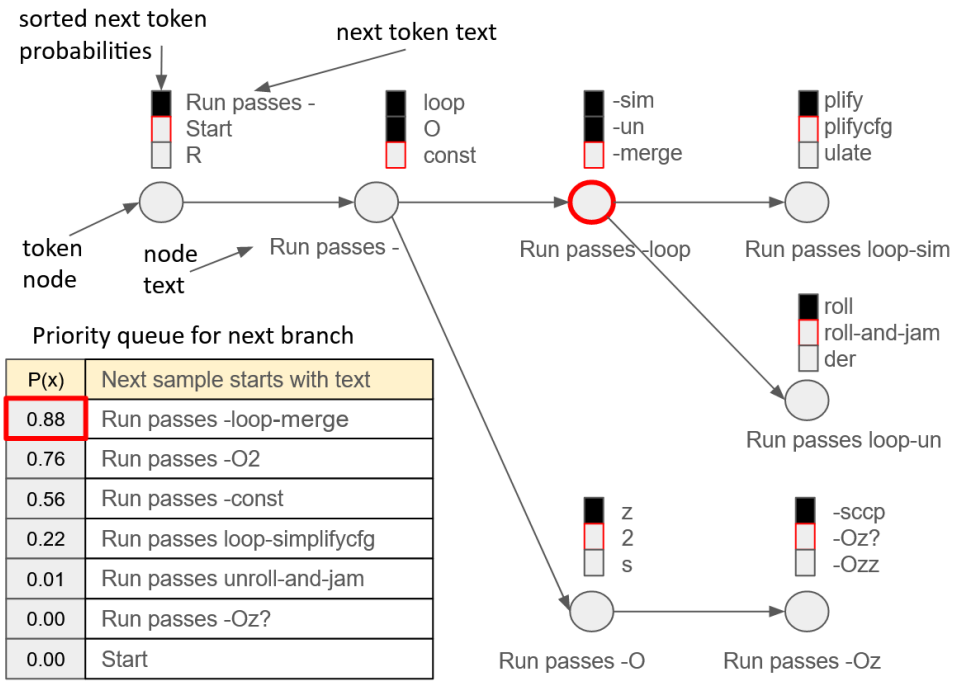
2 Priority Sampling of the LLM

At a high level, the Priority Sampling algorithm operates by augmenting a search tree and determining which unexplored path to expand next, based on the model's highest confidence. Our idea is simple. Always focus the search towards the most interesting direction based on previous samples, rather than determining this in advance. Additionally, avoid sample repetition, which decreases sampling power.

We sketch the algorithm for Priority Sampling in Figure 2. For the first sample, the model is equivalent to Greedy Decoding, but with an important addition. With each generation, it saves top K alternative tokens together with their predecessors in the priority queue with the token's probability as a metric. Once the sample is fully generated, we can quickly find the token prefix with the highest probability and expand the search tree from there. Since we add unexpanded tokens to the queue only once, each new sample will be unique. Additionally, we need the same number of inferences as the number of tokens generated in the search tree.

Going into more detail, the Algorithm 1 defines priority_queue and token_mask, which will be used for determining the best tokens prefix-sequence to expand and steer token generation to that point. Since we know the number of samples we generate, we can fix the length of priority_queue and set token_mask length to the generation length of the model.

With two *for* loops we iterate through sample space and token generation for each sample while keeping track of the previously generated tokens for a given sample. To determine the next token, we either follow the sequence from token_mask until we come to the branching point or expand the search tree by applying the inference.



P(x)	Next sample starts with text
0.88	Run passes -loop-merge
0.76	Run passes -O2
0.56	Run passes -const
0.22	Run passes loop-simplifycfg
0.01	Run passes unroll-and-jam
0.00	Run passes -Oz?
0.00	Start

Figure 2: Priority Sampling tree expansion. Each node contains a token generated by inference and the probabilities of the next potential tokens. In the first sample, we create a branch from the root to the end-of-sequence (EOS) token and put all valid potential tokens with their probabilities in the priority queue. For every next step, branch the token that had the highest probability and generate that branch until the EOS.

With inference, we get the probability distribution of tokens, from which we choose the *K* tokens with the highest probabilities. An important addition here is that we exclude all tokens that don't satisfy the regular expression we define when combined with previous tokens. This can be done in constant time by using a finite state machine as described in the previous work [28]. This technique enables us to steer generation only towards legal format which is particularly useful for code generation.

Once we select the best tokens, we expand the search tree directly with the best token, while putting the rest of the tokens on the queue. We repeat this until we finalize the generation of the current sample. After all potential tokens for expansions were saved to the queue, we update *token_mask* with the token prefix with the highest probability. Finally, we use the token prefix to locate the node that needs to be expanded and start the generation of the new sample from there.

Priority Sampling has the algorithmic complexity of $O(T*(inference + Klog(V)))$, where T is the number of generated tokens, K is the number of top-k samples we consider and V is vocabulary size. In practice, this is similar to Nucleus Sampling since the cost of inference is much higher than $Klog(V)$ and Priority Sampling reuses the inferences for the samples with the same prefix.

Additionally, the memory requirements are significantly reduced by keeping the size of the priority queue constant, equal to the number of samples we generate. This way we avoid saving the probabilities of all tokens in the vocabulary for each node in the search tree while ensuring that there will be enough candidates for branching the search tree.

Algorithm 1 Priority Sampling

```
priority_queue ← queue()
token_mask ← list()
sample_tokens ← list()
for sample : range(samples) do
    generated_tokens = list()
    for pos : range(generation_length) do
        if pos < len(token_mask) then
            next_token ← token_mask[pos]
        else
            logits ← inference(generated_tokens)
            best_tokens ← choose_best_tokens(logits, generated_tokens, regex)
            next_probability, next_token ← best_tokens[0]
            for probability, token : best_tokens[1:] do
                priority_queue.push(probability, generated_tokens + [token])
            end for
        end if
        generated_tokens.append(next_token)
    end for
    sample_tokens.append(generated_tokens)
    token_mask ← priority_queue.pop()
end for
return sample_tokens
```

3 Evaluation

We evaluate the Priority Sampling technique on the task of generating efficient LLVM optimization passes with LLM that reduce code size [7]. First, we train the 7B parameter model with Llama2 architecture for 30,000 steps on 64 V100s for a total training time of 620 GPU days. The training dataset consists of 1M LLVM IR labeled with the LLVM optimization sequence found by autotuner. To generate a label for each example, the autotuner spends 13 minutes exploring 37,424 optimization passes on average. Finally, we autotune 50k unseen test examples for 13 minutes for a total improvement of 4.98% over -Oz.

To evaluate the effectiveness of the Priority Sampling, we compare it to the Random Sampling, Greedy Decoding, and the Nucleus Sampling for 100 steps. Random Sampling evaluates random 100 optimization passes and for each sample calculates the best optimization pass so far. Greedy Decoding generates an optimization sequence by deterministically predicting the next token with the highest probability. For the Nucleus Sampling, we evaluate the model for temperature in the range {0.2, 0.4, 0.6, 0.8, 1, 1.2, 1.4, 1.6}. We found that for our problem and model architecture, temperature 1.2 is the most effective under 20 samples, while temperature 1.4 is the most effective for more than 20 samples.

We present the comparisons in Figure 3. Priority Sampling outperforms Random Sampling, Greedy Decoding, and Nucleus Sampling for any number of samples. Moreover, Priority Sampling is much more sample efficient than Nucleus Sampling achieving even the performance of the autotuner in 30 steps. Increasing the performance of the original model from 2.87% to over 5% with just 30 samples means that a significant part of knowledge is accessible by expanding the search tree wisely.

This is an astonishing result since the autotuner was trained to mimic the behavior of the autotuner, and not to outperform it. Since the autotuner operates on the complex set of LLVM optimizations tied to the input program's structure, our model seems to generalize from these patterns and combines them in a novel way on the unseen programs, which results in higher performance.

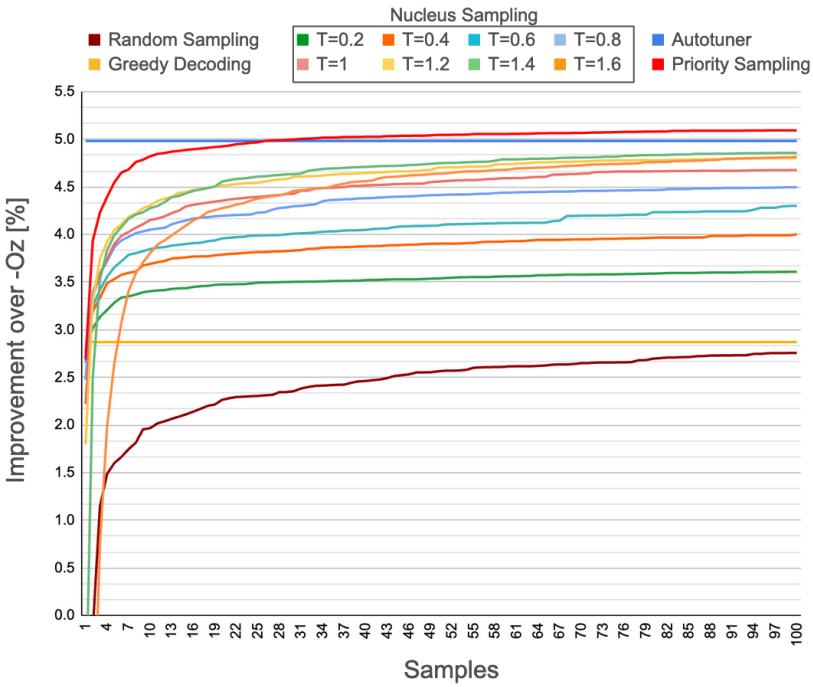


Figure 3: Average improvement in code size over -Oz optimization on 50k unseen test examples. Autotuner spends 760s for optimizing each example and sets the labels for LLM fine-tuning [7]. Greedy Decoding, Nucleus Sampling, and Priority Sampling utilize the fine-tuned model. Random Sampling selects 100 random flags for each sample. Priority Sampling outperforms all previous methods including autotuner which was used for labeling.

4 Ablations

For ablation (Table 1) we show how the performance of the program changes if:

1. We don't use regular expression filtering,
2. We use the geometric mean of probabilities of previously generated tokens as the metric for the priority queue,
3. We constrain the expansion for each node to 3 and 5.

If we don't enforce regular expression generation, the generated sampling tree will have higher probabilities, but generated samples could lead to invalid generation. To address this, we apply an additional pass that removes all invalid optimization passes and defaults to -Oz if all passes are illegal. For 1 and 100 samples this technique is beneficial, while enforcing regular expressions outperforms slightly non-constrained versions otherwise.

Next, we evaluate using the geometric mean as the metric for the priority_queue. This could be an interesting idea since the probability of the next token is highly biased with few previous tokens. For example prefix *-mem2* will put high probability to token *reg*, independently if *-mem2reg* is a good optimization to apply. On the other hand, calculating geometric mean doubles memory requirements since we need to store probability with each generated token. We found that this doesn't have a significant influence on the final performance.

Finally, we evaluate the performance of the method when the branching size is constraining. This idea focuses on increasing sample diversity and avoiding the generation of many nodes with the same prefixes. For a given prefix, the first few samples should be enough to finalize the optimization strategy, while we should use other samples to explore the alternatives. Our results suggest that there is some benefit of constraining the branching factor to 5 for our problem, but not significant.

	Improvement over -Oz [%]					
Method	Sample 1	Sample 3	Sample 5	Sample 10	Sample 30	Sample 100
Random Sampling	-12.56%	1.15%	1.60%	1.97%	2.36%	2.76%
Temp0	2.87%	-	-	-	-	-
Temp1.2	1.80%	3.74%	4.05%	4.31%	4.61%	4.80%
Temp1.4	-1.19%	3.52%	3.99%	4.28%	4.63%	4.86%
Temp1.6	-10.06%	0.75%	2.65%	3.81%	4.46%	4.82%
Autotuner	4.98%					
Priority Sampling (PS)	2.69%	**4.23%**	4.55%	4.82%	**5.00%**	5.09%
PS (no regex)	**3.17%**	4.18%	4.41%	4.64%	4.93%	**5.12%**
PS (max_branch 3)	2.62%	4.22%	4.56%	4.83%	4.99%	5.09%
PS (max_branch 5)	2.62%	4.22%	**4.61%**	**4.85%**	4.99%	5.09%
PS geometric (PSG)	2.68%	4.17%	4.45%	4.75%	4.96%	5.07%
PSG (max_branch 3)	2.62%	4.17%	4.52%	4.77%	4.98%	5.11%
PSG (max_branch 5)	2.62%	4.17%	4.56%	4.80%	4.98%	**5.12%**

Table 1: Experimental results and ablation experiments of Priority Sampling. Evaluation includes the improvement of Random Sampling, Nucleus Sampling, and Autotuner over the compiler (default -Oz optimization). Ablation evaluates the use of the regular expression, constraining branching factor, and using the geometric mean as the priority metric in Priority Sampling.

5 Related Work

Stochastic Methods introduce noise in the process of selecting the next token, resulting in increasing diversity of generation. Top-k Sampling narrows the choice of the next token to the top k most probable tokens [9]. Nucleus Sampling [11] eliminates the low-probability tail of the distribution and preserves diversity by sampling from tokens whose sum is larger than *top-p=0.95* probability. Mirostat [4] provides a mechanism for controlling the perplexity of the generated text. Noisy Parallel Approximate Decoding [6] inserts unstructured Gaussian noise in each layer resulting in diverse samples. Unlike Stochastic Methods, Priority Sampling guarantees diverse set of samples deterministically.

Beam Methods manipulate the expansion mechanisms by constructing a search tree and focusing the direction of exploration. Diverse Beam Search [25] decodes diverse lists by dividing the beam budget into groups and enforcing diversity between groups of beams. Determinantal beam search [18] defines beam search as an iterative subdeterminant maximization problem and encodes the diversity as an optimization metric. Conditional Poisson Stochastic Beam Search [19] sample K candidates without replacement according to the conditional Poisson sampling design, resulting in a low-variance consistent estimator.

Instead of shaping reward function based on diversity, Stochastic Beam Search [12] uses Gumbel-Max trick [10] to sample top-k tokens with the highest probabilities in differentiable manner. The Gumbel-Max Trick involves adding Gumbel-distributed noise to the logits (likelihood scores), after which we apply the softmax and select top-k candidates. By applying this procedure recursively for each next token, Stochastic Beam Search returns a fixed-size batch of samples.

Although Stochastic Beam Search and Gumbel top-k sampling guarantee a different output for each beam element, they are not easy to parallelize. Arithmetic Sampling [26] solves this problem by first sampling N numbers from uniform(0, 1) distribution and then recursively expanding tokens whose probability interval includes any of the given numbers. This method guarantees a diverse set of samples with high probability which is easy to parallelize, although it may include duplicates.

Unique Randomizer [24] incrementally samples sequence models while guaranteeing the uniqueness of each sample. Unique Randomizer constructs a trie to keep track of probability distribution mass for each token. Every time a sample is fully generated it subtracts it's probability from parent nodes, guaranteeing that the sample will not be selected in the future.

In our work, we extend further the idea of a Unique Randomizer with few key differences. First, we augment the trie of generated tokens with a priority queue that keeps the probabilities of each token together with its prefix. This enables us to quickly and deterministically find the node in the

trie that needs to be expanded next while avoiding inferences for the prefix tokens. Additionally, we expand the priority queue with tokens that together with prefixes satisfy the regular expression that we provide, keeping the size of the trie minimal.

Controlable text generation. Zhang et. al [31] describes a comprehensive list of challenges for controllable text generation. Unlikelyhood training [27] introduces a novel training objective that explicitly decreases the probability of unlikely generations. Lagutin et. al [14] proposed Implicit Unlikelyhood Training which uses policy gradient reinforcement learning to reduce the repetition of generated text. Willard et. al [28] introduced an efficient guiding algorithm for guiding inference of LLMs based on regular expressions. This research direction is orthogonal to our approach and could be used together with Priority Sampling.

6 Limitations

Priority Sampling provides an efficient way to get high-quality diverse samples, but it comes with a few limitations. The current implementation is inherently sequential. To decide what branch needs to be expanded next, it needs to construct an augmented search tree. One way to parallelize Priority Sampling would be to treat the priority queue as a task generator, from which threads take the next branching position whenever they are idle. Second, Priority Sampling needs to find the top N next tokens that match the regular expression, which is more time-consuming than sampling methods such as Unique Randomizer [24] and Arithmetic Sampling [26]. This is however necessary step for generating samples in order.

7 Conclusion

We present Priority Sampling, a simple inference technique that provides a deterministic and controllable way of generating unique samples for which LLM is the most confident. Priority Sampling is much more sample efficient than widely used Nucleus Sampling and outperforms it for any number of samples.

We evaluate our model on an LLVM pass ordering task, in which our model is trained to predict the optimization passes found by a long-running autotuner. Our model was able to boost the performance of the original model from 2.87% to 5% improvement over default optimization -Oz in 30 steps and even more to outperform the autotuner which was used to generate training labels.

This is an astonishing result that supports the argument that LLMs store a large amount of knowledge that is accessible with the clever expansion of the search tree. Additionally, Priority Sampling includes support for regular expression generation that provides a controllable and structured exploration process.

References

[1] Baleegh Ahmad, Shailja Thakur, Benjamin Tan, Ramesh Karri, and Hammond Pearce. Fixing hardware security bugs with large language models. *arXiv preprint arXiv:2302.01215*, 2023.

[2] Loubna Ben Allal, Raymond Li, Denis Kocetkov, Chenghao Mou, Christopher Akiki, Carlos Munoz Ferrandis, Niklas Muennighoff, Mayank Mishra, Alex Gu, Manan Dey, Logesh Kumar Umapathi, Carolyn Jane Anderson, Yangtian Zi, Joel Lamy Poirier, Hailey Schoelkopf, Sergey Troshin, Dmitry Abulkhanov, Manuel Romero, Michael Lappert, Francesco De Toni, Bernardo García del Río, Qian Liu, Shamik Bose, Urvashi Bhattacharyya, Terry Yue Zhuo, Ian Yu, Paulo Villegas, Marco Zocca, Sourab Mangrulkar, David Lansky, Huu Nguyen, Danish Contractor, Luis Villa, Jia Li, Dzmitry Bahdanau, Yacine Jernite, Sean Hughes, Daniel Fried, Arjun Guha, Harm de Vries, and Leandro von Werra. SantaCoder: don't reach for the stars! *arXiv:2301.03988*, 2023.

[3] Jordi Armengol-Estapé and Michael FP O'Boyle. Learning C to x86 Translation: An Experiment in Neural Compilation. *arXiv:2108.07639*, 2021.

[4] Sourya Basu, Govardana Sachitanandam Ramachandran, Nitish Shirish Keskar, and Lav R Varshney. Mirostat: A neural text decoding algorithm that directly controls perplexity. *arXiv preprint arXiv:2007.14966*, 2020.

[5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating Large Language Models Trained on Code. *arXiv:2107.03374*, 2021.

[6] Kyunghyun Cho. Noisy parallel approximate decoding for conditional recurrent language model. *arXiv preprint arXiv:1605.03835*, 2016.

[7] Chris Cummins, Volker Seeker, Dejan Grubisic, Mostafa Elhoushi, Youwei Liang, Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Kim Hazelwood, Gabriel Synnaeve, et al. Large language models for compiler optimization. *arXiv preprint arXiv:2309.07062*, 2023.

[8] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In *ISSTA*, 2023.

[9] Angela Fan, Mike Lewis, and Yann Dauphin. Hierarchical neural story generation. *arXiv preprint arXiv:1805.04833*, 2018.

[10] Emil Julius Gumbel. Statistical theory of extreme valuse and some practical applications. *Nat. Bur. Standards Appl. Math. Ser. 33*, 1954.

[11] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. *arXiv preprint arXiv:1904.09751*, 2019.

[12] Wouter Kool, Herke Van Hoof, and Max Welling. Stochastic beams and where to find them: The gumbel-top-k trick for sampling sequences without replacement. In *International Conference on Machine Learning*, pages 3499–3508. PMLR, 2019.

[13] Marie-Anne Lachaux, Baptiste Roziere, Lowik Chanussot, and Guillaume Lample. Unsupervised Translation of Programming Languages. *arXiv:2006.03511*, 2020.

[14] Evgeny Lagutin, Daniil Gavrilov, and Pavel Kalaidin. Implicit unlikelihood training: Improving neural text generation with reinforcement learning. *arXiv preprint arXiv:2101.04229*, 2021.

[15] Dong Li, Ruoming Jin, Jing Gao, and Zhi Liu. On sampling top-k recommendation evaluation. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2114–2124, 2020.

[16] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Mishig Davaadorj, Joel Lamy-Poirier, João Monteiro, Oleh Shliazhko, Nicolas Gontier, Nicholas Meade, Armel Zebaze, Ming-Ho Yee, Logesh Kumar Umapathi, Jian Zhu, Benjamin Lipkin, Muhtasham Oblokulov, Zhiruo Wang, Rudra Murthy, Jason Stillerman, Siva Sankalp Patel, Dmitry Abulkhanov, Marco Zocca, Manan Dey, Zhihan Zhang, Nour Fahmy, Urvashi Bhattacharyya, Wenhao Yu, Swayam Singh, Sasha Luccioni, Paulo Villegas, Maxim Kunakov, Fedor Zhdanov, Manuel Romero, Tony Lee, Nadav Timor, Jennifer Ding, Claire Schlesinger, Hailey Schoelkopf, Jan Ebert, Tri Dao, Mayank Mishra, Alex Gu, Jennifer Robinson, Carolyn Jane Anderson, Brendan Dolan-Gavitt, Danish Contractor, Siva Reddy, Daniel Fried, Dzmitry Bahdanau, Yacine Jernite, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. StarCoder: may the source be with you! *arXiv:2305.06161*, 2023.

[17] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with AlphaCode. *Science*, 378(6624), 2022.

[18] Clara Meister, Martina Forster, and Ryan Cotterell. Determinantal beam search. *arXiv preprint arXiv:2106.07400*, 2021.

[19] Clara Meister, Tiago Pimentel, Gian Wiher, and Ryan Cotterell. Typical decoding for natural language generation. *arXiv preprint arXiv:2202.00666*, 2022.

[20] OpenAI. GPT-4 Technical Report. *arXiv:2303.08774*, 2023.

[21] Shauli Ravfogel, Yoav Goldberg, and Jacob Goldberger. Conformal nucleus sampling. *arXiv preprint arXiv:2305.02633*, 2023.

[22] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. Code Llama: Open Foundation Models for Code. *arXiv:2308.12950*, 2023.

[23] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. Adaptive Test Generation Using a Large Language Model. *arXiv:2302.06527*, 2023.

[24] Kensen Shi, David Bieber, and Charles Sutton. Incremental sampling without replacement for sequence models. In *International Conference on Machine Learning*, pages 8785–8795. PMLR, 2020.

[25] Ashwin K Vijayakumar, Michael Cogswell, Ramprasath R Selvaraju, Qing Sun, Stefan Lee, David Crandall, and Dhruv Batra. Diverse beam search: Decoding diverse solutions from neural sequence models. *arXiv preprint arXiv:1610.02424*, 2016.

[26] Luke Vilnis, Yury Zemlyanskiy, Patrick Murray, Alexandre Tachard Passos, and Sumit Sanghai. Arithmetic sampling: parallel diverse decoding for large language models. In *International Conference on Machine Learning*, pages 35120–35136. PMLR, 2023.

[27] Sean Welleck, Ilia Kulikov, Stephen Roller, Emily Dinan, Kyunghyun Cho, and Jason Weston. Neural text generation with unlikelihood training. *arXiv preprint arXiv:1908.04319*, 2019.

[28] Brandon T Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv e-prints*, pages arXiv–2307, 2023.

[29] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. Automated program repair in the era of large pre-trained language models. In *Proceedings of the 45th International Conference on Software Engineering (ICSE 2023). Association for Computing Machinery*, 2023.

[30] Yuxi Xie, Kenji Kawaguchi, Yiran Zhao, Xu Zhao, Min-Yen Kan, Junxian He, and Qizhe Xie. Self-evaluation guided beam search for reasoning. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.

[31] Hanqing Zhang, Haolin Song, Shaoyu Li, Ming Zhou, and Dawei Song. A survey of controllable text generation using transformer-based pre-trained language models. *ACM Computing Surveys*, 56(3):1–37, 2023.